DUNDALK INSTITUTE OF TECHNOLOGY

DK_ICSOP_7 - B.Sc. in Computing Systems and Operations

DK_ICCSO_8 – B.Sc. (Hons) in Computing Systems and Operations

Stage 3 - Semester 5

JANUARY 2026

COMP I8064 - Data Structures

Internal Examiner(s): Dr Michelle Graham

External Examiner(s): xxxxxxx

Date:

Time:

Duration:

Venue:

No. Pages:

Instructions

Answer QUESTION ONE [40 marks] and any other TWO questions [30 marks each].

PLEASE NOTE: Exam Paper printed DOUBLE SIDED.

DO NOT TURN THIS PAGE UNTIL INSTRUCTED

You MUST attempt this question.

Question 1) Data Structure Fundamentals

(a) Data structures are designed to store and operate on data within a program.

(i) What is an Abstract Data Type (ADT)? What is the benefit of separating a data structure's ADT from its implementation?

(5 marks)

(ii) Formally specify the ADT for a *List* structure. Your ADT should include all fundamental operations for a *List* and explain the need for their presence.

(10 marks)

(b) Generics are a programming feature designed to make data structures more flexible and reusable.

(i) Explain what is meant by a generic data structure and how it differs from a non-generic data structure. Provide an example to illustrate your answer.

(4 marks)

(ii) Outline and explain the key code-level elements required to implement a generic data structure in Java. In your answer you should refer to both **node-based** structures (e.g. linked lists) and **array-based** structures (e.g. dynamic arays).

(5 marks)

(iii) Explain one advantage and one potential drawback to implementing a data structure as a **generic** version vs implementing as a **type-specific** (non-generic) version.

(4 marks)

(c) Your team has been asked to write unit tests for a PriorityQueue. The PriorityQueue is built to hold Task objects, is unbounded, and duplicate values are allowed. Null values should not be allowed. The Task class contains a compareTo method that ranks instances in order of priority (with highest being 5 and lowest being 1).

You have been assigned the following method:

- public boolean add(String s)

Outline **three** test cases that should be included as unit tests for the above method. For each test case, explain its necessity, what assertions should be included and why they are needed. You may assume the existence of a get() method to allow direct access to specific points in the PriorityQueue

(12 marks)

Choose any TWO questions from questions 2, 3 and 4

Question 2) Stacks and Queues

(a) Explain the concept of "fail fast" programming and how it can be applied to the implementation of a **Stack** data structure. Include one or more examples of how a Stack can detect and respond to invalid operations at runtime.

(5 marks)

(b) A PriorityQueue is a structure in which the data is stored in order of its relative priority.

(i) Write and explain the **pseudocode** for an *add(Job j)* method that inserts a Job object into an unbounded PriorityQueue implemented as a **singly-linked node-based List**.

Each Job has three attributes:

- priority - an integer between 1 & 5 (5 is highest priority)
- description - a string describing the Job
- timestamp – the time when the Job was created (a LocalDateTime)

The PriorityQueue should stay sorted in descending order of Job priority (highest priority first). Your answer should include pseudocode for the insertion logic, as well as an explanation of what each step does and why it is needed.

*Note: Your method should be done as **pseudocode**, not actual code. Pseudocode and explanation are worth equal marks.*

(15 marks)

(ii) Evaluate whether a singly-linked node-based List is an efficient structure for implementing a PriorityQueue. If not, suggest a more suitable alternative and justify your choice with reference to worst-case performance and operation efficiency.

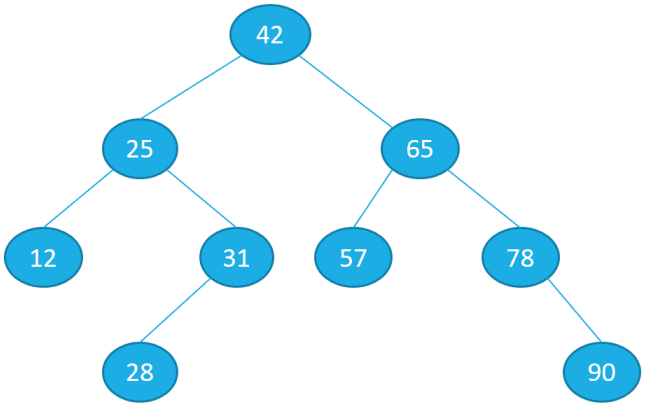
(10 marks)

Question 3) Binary Search Trees

(a) Explain the concept and structure of the *Binary Search Tree (BST)* data structure. List and describe the fundamental operations that can be performed on a *Binary Search Tree*, outlining their general purpose and time complexity.

(8 marks)

(b) The following is a depiction of a Binary Search Tree (BST).



(i) What is meant by the *traversal* of a BST tree? Explain the three orders of tree traversal, illustrating your answer with the result of the tree's traversal under each approach.

(14 marks)

(ii) Explain what is meant by an **unbalanced** Binary Search Tree (BST). In your answer, describe one scenario in which a BST may become unbalanced, and discuss how this imbalance affects the performance of common BST operations such as search, insertion, and deletion. Your explanation should include reference to Big-O time complexity.

(8 marks)

Question 4) Analysis of Data Structures

A company operates a real-time analytics platform that collects millions of event records per hour. Each event has:

- A unique eventID (String)
- A timestamp (LocalDateTime)
- Associated metadata (key–value information)

The system must:

- Efficiently **store** and **access** these event records.
- Support **fast existence checks** to see if a given eventID has already been processed.
- Occasionally retrieve events in **sorted order** by eventID for reporting.
- Use **minimal memory** and minimize performance loss caused by hash collisions.

The development team is considering three data structures:

- **HashSet**
- **ListSet** (implemented as a dynamic array / array-based list)
- **HashMap** (with an unspecified collision-handling approach)

(a) Compare and contrast HashSet, ListSet, and HashMap data structures with respect to:

- Internal structure & organization of data (hash-based vs node-based vs array-based)
- Ordering and uniqueness behaviour
- Average & worst-case time complexity for insertion, lookup & deletion
- Memory overhead

Use examples where appropriate.

(24 marks)

(b) Recommend **the most suitable data structure** for this analytics platform scenario. Justify your choice by linking its key features to the scenario's requirements, specifically addressing:

- Expected lookup and insertion performance
- The impact of hash collisions
- If the need for sorted order justifies any performance trade-offs

(6 marks)